

Guía Completa sobre JSX en React

¿Qué es JSX?

JSX (JavaScript XML) es una extensión de sintaxis para JavaScript que permite escribir código similar a HTML dentro de JavaScript. Es una de las principales características de React y facilita la creación de interfaces de usuario al combinar la estructura visual y la lógica en un solo lugar.

Aunque JSX se parece mucho a HTML, no es exactamente lo mismo. Es transformado por herramientas como Babel en código JavaScript puro que React puede entender.

Ejemplo simple de JSX:

```
const element = <h1>¡Hola, mundo!</h1>;
```

Este código JSX se convierte en JavaScript equivalente:

```
const element = React.createElement('h1', null, '¡Hola, mundo!');
```

Ventajas de JSX

1. **Sintaxis declarativa:** Hace que sea más fácil visualizar la estructura del componente.
 2. **Integración con JavaScript:** Puedes escribir código JavaScript directamente dentro de JSX usando llaves {}.
 3. **Mejor experiencia de desarrollo:** Ofrece mejores mensajes de error y compatibilidad con herramientas como linters y editores.
-

Reglas básicas de JSX

1. Código JSX dentro de un solo contenedor

JSX siempre debe devolver un único elemento contenedor. Por ejemplo:

```
// Incorrecto
return (
  <h1>¡Hola!</h1>
  <p>Bienvenidos al curso de React.</p>
);
```

```
// Correcto
```

```
return (  
  <div>  
    <h1>¡Hola!</h1>  
    <p>Bienvenidos al curso de React.</p>  
  </div>  
);
```

Si no deseas agregar un elemento contenedor como <div>, puedes usar **Fragmentos**:

```
import React from 'react';  
  
return (  
  <>  
    <h1>¡Hola!</h1>  
    <p>Bienvenidos al curso de React.</p>  
  </>  
);
```

2. Interpolación de JavaScript

Puedes insertar código JavaScript dentro de JSX usando llaves {}:

```
const nombre = 'María';  
return <h1>¡Hola, {nombre}!</h1>;
```

Dentro de las llaves puedes realizar operaciones:

```
const x = 5;  
const y = 10;  
return <p>La suma es {x + y}</p>;
```

3. Atributos en JSX

Los atributos en JSX se escriben de forma similar a los de HTML, pero con algunas diferencias:

- Se usa **camelCase** para los nombres de atributos (e.g., className en lugar de class).
- Los valores de los atributos pueden ser cadenas de texto, variables o expresiones JavaScript:

```
const url = 'https://reactjs.org';  
return <a href={url} target="_blank">Ir a React</a>;
```

4. className en lugar de class

En lugar de usar el atributo class como en HTML, en JSX se usa className porque class es una palabra reservada en JavaScript:

```
return <div className="mi-clase">Contenido</div>;
```

5. Estilo en línea

Para aplicar estilos en línea, se usa un objeto de JavaScript con propiedades en camelCase:

```
const estilos = {  
  color: 'blue',  
  fontSize: '20px',  
};  
return <p style={estilos}>Texto con estilo</p>;
```

// O directamente en el atributo style:

```
return <p style={{ color: 'red', fontWeight: 'bold' }}>Texto rojo y negrita</p>;
```

6. Comentarios en JSX

Los comentarios se escriben dentro de llaves:

```
return (  
  <div>  
    { /* Este es un comentario en JSX */ }  
    <h1>Hola</h1>  
  </div>  
);
```

JSX y funciones

Puedes usar funciones para renderizar contenido dinámico:

```
function saludo(nombre) {  
  return <h1>¡Hola, {nombre}!</h1>;  
}  
  
return (  
  <div>  
    {saludo('Carlos')}  
  </div>  
);
```

Listas y bucles en JSX

Para renderizar listas, puedes usar el método map:

```
const frutas = ['Manzana', 'Banana', 'Cereza'];
return (
  <ul>
    {frutas.map((fruta, index) => (
      <li key={index}>{fruta}</li>
    ))}
  </ul>
);
```

- **key** es un prop especial que React utiliza para identificar elementos únicos en listas. Ayuda a mejorar el rendimiento al actualizar listas.

Manejo de listas anidadas

Si tienes listas dentro de listas, asegúrate de usar claves únicas en cada nivel:

```
const categorias = [
  { nombre: 'Frutas', items: ['Manzana', 'Banana'] },
  { nombre: 'Verduras', items: ['Zanahoria', 'Brócoli'] },
];

return (
  <div>
    {categorias.map((categoria, catIndex) => (
      <div key={catIndex}>
        <h3>{categoria.nombre}</h3>
        <ul>
          {categoria.items.map((item, itemIndex) => (
            <li key={itemIndex}>{item}</li>
          ))}
        </ul>
      </div>
    ))}
  </div>
);
```

Condicionales en JSX

Puedes mostrar u ocultar elementos basándote en condiciones:

1. Operador ternario

```
const esAdmin = true;
return (
  <div>
    {esAdmin ? <p>Eres administrador</p> : <p>Eres usuario</p>}
  </div>
);
```

2. Operador AND (&&)

```
const mostrarMensaje = true;
return (
  <div>
    {mostrarMensaje && <p>Este mensaje solo aparece si mostrarMensaje es true</p>}
  </div>
);
```

3. Funciones auxiliares

Puedes usar funciones para manejar condiciones más complejas:

```
function renderizarMensaje(esAdmin) {
  if (esAdmin) {
    return <p>Eres administrador</p>;
  }
  return <p>Eres usuario</p>;
}

return <div>{renderizarMensaje(true)}</div>;
```

4. Switch en JSX

Si necesitas manejar varias condiciones, puedes usar un switch:

```
function renderizarEstado(estado) {  
  switch (estado) {  
    case 'cargando':  
      return <p>Cargando...</p>;  
    case 'exito':  
      return <p>¡Operación exitosa!</p>;  
    case 'error':  
      return <p>Error al cargar.</p>;  
    default:  
      return <p>Estado desconocido.</p>;  
  }  
}  
  
return <div>{renderizarEstado('exito')}</div>;
```

Eventos en JSX

Puedes manejar eventos como onClick, onChange, etc., directamente en JSX:

1. Manejadores de eventos

```
function handleClick() {  
  alert('Hiciste clic');  
}  
  
return <button onClick={handleClick}>Haz clic aquí</button>;
```

2. Eventos con parámetros

Usa una función flecha para pasar parámetros a los eventos:

```
function saludar(nombre) {  
  alert(`Hola, ${nombre}`);  
}  
  
return <button onClick={() => saludar('Carlos')}>Saludar</button>;
```

3. Eventos del teclado

```
function handleKeyPress(event) {  
  if (event.key === 'Enter') {  
    alert('Presionaste Enter');  
  }  
}  
  
return <input onKeyPress={handleKeyPress} placeholder="Escribe algo..." />;
```

Advertencias comunes

1. **Solo un elemento contenedor:** JSX siempre necesita devolver un único elemento padre.
2. **No mezclar strings y código directamente:** Las expresiones JavaScript siempre deben ir entre {}.
3. **Uso correcto de key:** Asegúrate de proporcionar claves únicas al renderizar listas para evitar errores de rendimiento.
4. **Nombres de eventos en camelCase:** Los eventos como onclick en HTML deben escribirse como onClick en JSX.